Verifiable Circuit Wave Analysis

Timothy Welch and Andrew W. Appel

March 24, 2026

1 Introduction

A *rotating detonation engine* burns fuel (and oxidizer) in an annular channel, that is, in the narrow space between two concentric cylinders. Detonation waves travel clockwise or counterclockwise around the annulus, burning fuel faster (and therefore generating more thrust) than if detonation was not occurring.

But are the detonation waves stable and long-lived? With what speed and direction do they travel? Is there just one wave or several simultaneously? What is the shape of the wave? How much combustion that occurs is unrelated to the detonation waves? To answer these questions, Chacon, Feleo, and Gamba [1] aimed a camera at the back of the engine, imaging the brightness of chemiluminescence at 101 equally spaced points around the cylinder, every 18 microseconds, for durations of 1.5 seconds (thus, about 82,000 images). Under the hypothesis that in any 20ms timeframe there are just a few detonation waves present, they used Fourier analysis to estimate the amplitude and angular velocity of the detonation waves. This analysis is called *circuit wave analysis* (CWA). CWA takes into account that the wave velocity may change (somewhat) over the timeframe, which allows it to get significantly better results than a simple Fourier analysis that would assume a constant velocity. However, although Chacon *et al.*'s CWA deduces important properties of the circuit waves, it does not guarantee that the waves it identifies account for all the activity in the combustion annulus, nor does it *describe* the gradual changes in velocity of the waves, nor does it describe the shape of the waveform.

Based on Chacon *et al.*'s CWA, we have designed and implemented a *Verified CWA* (VCWA), that produces a description D of the wave activity seen by the imaging device along with an accuracy tolerance τ. It makes this formally verifiable[1] guarantee: VCWA's description of the waves accounts for, within the accuracy tolerance, *all* the brightness variation seen by the imager. In any one run of VCWA, the accuracy tolerance may be small (indicating that it found a fairly complete description of the activity) or large (indicating either that the brightness variations could not be described by just a few waves or that it could not identify those waves).

[1] soon to be formally verified

Either way, VCWA makes a verifiable claim: if you synthesize the waveforms described by D and subtract the original image sequence, the 1-norm of the result will be $\leq \tau$. To verify this claim, run a simple program that performs this synthesis and subtraction.

That is, we have an *analyzer,* that uses Fourier analysis and optimization algorithms to produce a *hypothesis* (D, τ), and a *verifier*. The analyzer is implemented in MATLAB. The verifier program is much simpler than the analyzer—since all it does is synthesize and sum some waveforms—and we have implementations in MATLAB and in C.

One can even apply *formal verification* to VCWA. Formal verification, in this context, is a *proof* in mathematical logic that a program satisfies its mathematical specification. Since any such proof must be with respect to the minute details of the programming language's formal semantics, there are far too many details in such proofs to reliably perform or check such proofs by hand. Therefore, formal program verification usually uses software *proof checkers* to check that every step of such proofs is valid, and *proof assistants* and/or *automated theorem provers* to automate many steps of the proof.

We formally verify the verifier, not the analyzer. We believe the analyzer is correct, but it need not be *proved* correct, since any hypothesis it proposes can be independently checked by the verifier. Furthermore, this arrangement allows continued improvements of the analyzer's (to describe a set of waves that more accurately account for the input data) without altering the verifier or its proof.

2 Specification

The *input data* is a time series $X : \mathbb{R}^{N_{\text{fr}}} \times \mathbb{R}^{N_{\text{b}}}$ of N_{fr} *frames* each containing N_{b} *brightness* values. Typically N_{fr} might be approximately 10^5 and (in the experimental apparatus used) $N_{\text{b}} = 101$. The brightness values are understood as equally spaced around the circumference of a circle, so that points i and $(i+1) \mod N_{\text{b}}$ are adjacent in (angular) space. Between frame k and frame $k+1$ there is time interval δ, typically 20 microseconds. Thus, $X_{k,i}$ represents the brightness at time $k\delta$, angle $\theta(i) = \frac{2\pi}{N_{\text{b}}} i$.

Hypothesis, first attempt. One might hypothesize that brightness $X_{k,i}$ is the sum of W waves, where the wth wave $(0 \leq w < W)$ is described by a velocity v_w, amplitude A_w, phase $\theta_{w,0}$, and shaped like a Gaussian distribution of width σ_w. Therefore we synthesize

$$S^{\text{naive}}_{k,i} = \sum_{w=0}^{W-1} A_w \exp\left(-\frac{(\theta_{w,0} + v_w k\delta - \frac{2\pi}{N_{\text{b}}} i)^2}{2\sigma_w^2}\right). \tag{1}$$

However, this does not lead to an accurate estimation of X. The waves do not have constant amplitude, nor constant velocity, nor constant width. Instead, all three of these are slowly varying functions of time.[2]

[2] If these values were arbitrary chaotic functions of time, it would be inappropriate to speak of a "wave." The limitation to slowly varying functions is intended to produce a

Furthermore, we found that there is a *static background*. That is, each of the N_b points around the ring is consistently brighter or dimmer, regardless of t. This may be a calibration issue, or it may be that some parts of the ring are consistently hotter than others; either way, we can formulate a hypothesis with a static background $B : \mathbb{R}^{N_\mathrm{b}}$, which is also a slowly varying function of time.

Hypothesis, final form. For $0 \le k < N_\mathrm{fr}$ and $0 \le i < N_\mathrm{b}$, we explain $X_{k,i}$ as the sum of W waves plus a static background.

To represent that the wth wave's amplitude, width, velocity are *slowly varying* functions of time, we represent each of those functions by a time series over N_w equally spaced points, where $N_w < N_\mathrm{fr}$, spanning the time 0 to $(N_\mathrm{fr} - 1)\delta$. That is, for each j such that $0 \le j \le N_w$, the wave's amplitude at time $\frac{N_\mathrm{fr}\delta}{N_w}k$ is $A_{w,j}$, its width is $\sigma_{w,j}$, and its velocity is $v_{w,j}$, and its initial position is $\theta_{w,0}$.

Then we use linear interpolation to reconstruct the value at time $k\delta$. For example, the interpolation of A at time $k\delta$ is calculated as follows. Let $f = (N_\mathrm{fr} - 1)/(N_w - 1)$, the ratio between the real time range and the compressed index range. Let $j = \lfloor k/f \rfloor$ and $i + 1$ be the points in (the compressed) A between which we want to interpolate. Then the result is

$$\hat{A}_{w,k} = A_{w,j} + \frac{k - fj}{f}(A_{w,j+1} - A_{w,j}). \tag{2}$$

The velocity wave w is a slowly varying function of time $v_{w,j}$ where j ranges from 0 to $N_w - 1$. We interpolate as above to calculate $\hat{v}$, and we reconstruct the angular position of the wth wave at the kth time-step, as follows.

$$\theta_{w,k} = \begin{cases} \theta_{w,0} & \text{if } k = 0 \\ (\theta_{w,k-1} + \hat{v}_{w,k}\delta) \mod 2\pi & \text{if } k > 0 \end{cases} \tag{3}$$

The shape of wave w is assumed to be a Gaussian with width $\sigma_{w,k}$ at time $k\delta$, so the height of w at time-step k and position i is,

$$a_{w,k,i} = \hat{A}_{w,i} \exp(-\mathrm{wrap}(2\pi\frac{i}{N_\mathrm{b}} - \theta_{w,k})^2/(2\sigma^2)) \tag{4}$$

where $\mathrm{wrap}(\phi)$ range-reduces ϕ to a value between $-\pi$ and $+\pi$.

The static background is represented as a two-dimensional array $B_{j,i}$, for $0 \le j < N_\mathrm{B}$ and $0 \le i < N_\mathrm{b}$, where $N_\mathrm{B} < N_\mathrm{fr}$ to indicate that B is a slowly varying function of time. However, in reconstruction we do not use linear interpolation, just simple rounding to nearest. That is, the synthesized background at time k, position i is,

$$\hat{B}_{k,i} = B_{j,i} \qquad \text{where } j = \frac{N_\mathrm{B} - 1}{N_\mathrm{fr} - 1}k \tag{5}$$

Thus we define the synthesized brightness function as,

$$S_{k,i} = \hat{B}_{k,i} + \sum_{w=0}^{W-1} a_{w,k,i} \tag{6}$$

hypothesis that has some scientific interpretability for the engineer designing and measuring the combustion chamber.

Now we can calculate the accuracy of this hypothesis. The error ϵ is the sum of the differences at each point:

$$\epsilon = \sum_{k=0}^{N_{\mathrm{fr}}-1} \sum_{i=0}^{N_{\mathrm{b}}-1} |S_{k,i} - X_{k,i}| \tag{7}$$

Conclusion: To explain a dataset $(N_{\mathrm{fr}}, N_{\mathrm{b}}, X)$, a *hypothesis* is a tuple (D, τ) where $D = (B, (N_0, A_0, v_0, \sigma_0), \ldots (N_{w-1}, A_{w-1}, v_{w-1}, \sigma_{w_1}))$; where each A_w, v_w, and σ_w is in $\mathbb{R}^{N_w}$; and where the error $\epsilon = ||S(D) - X||_1$ is $\leq \tau$.

3 How to find a hypothesis

it ain't trivial. Tim will explain ...

4 How to verify a hypothesis

Listing 1 shows the C program of the verifier.

5 Proving correctness of the verifier

not done yet

References

[1] F. Chacon, A.D. Feleo, and M. Gamba. Secondary waves dynamics and their impact on detonation structure in rotating detonation combustors. *Shock Waves*, 31:675–702, 2021.

Listing 1: The verifier as a C program

```
#define L (2.0*M_PI)
double range2pi(double x) {        // calculate x mod 2π
  return x - L * floor (x / L);
}

double wrap(double x) {
  return range2pi(x+L/2.0)-L/2.0;
}

double interpolate(double f, double *y, int n, double k) {
     // equation (2)
     int j = floor(k/f);
     return y[j] + ((k - j*f) / f) * (y[j+1] - y[j]);
}

void synthesize(Data *data, double *S) {
  int Nw = data→num_waves, Nb = data→Nb, Nfr = data→Nfr,
    Nbg  = data→Nbg;
  double dt = data→dt;
  Wave *waves = data→waves;
  double theta [ MAX_WAVES ];
  int w,k,i;

  for (k = 0; k < Nfr; k++) {
    for (i = 0; i < Nb; i++) {
      double a = data→BG[i + (int)(k*((double)(Nbg - 1)) /
   (Nfr - 1))*Nb];  // equation (5)

      for (w = 0; w < Nw; w++) {
        int Nt = waves[w].Nt;
        double f = (double)(Nfr - 1) / (double)(Nt - 1);
        double A     = interpolate(f, waves[w].A, Nt, k);
     // equation (2)
        double sigma = interpolate(f, waves[w].sigma, Nt,
   k);

        if (k==0)
          theta[w] = waves[w].theta0;
        else if (i==0) {
          double v = interpolate(f, waves[w].v, Nt, k-1);
          theta[w] = range2pi (theta[w] + v * dt);
    // equation (3)
        }

        double dx = wrap(i * (L / Nb) - theta[w]);
        a += A * exp(-dx*dx / (2.0 * sigma * sigma));
   // equation (??)
      }
      S[i + k*Nb] = a;
} } }
```